UE 7 : Mobility and Smart Cities

Philippe Canalda

Philippe.Canalda@univ-fcomte.fr

Remember …

- 2 main objectives, a first one which is real-time organization of mobility (optimization, prediction) : course, Directed Work & Practical Work, a mini-project,
  a second one which is continuity of positionning and mobility (indoor <-> outdoor) : Practical Work and a mini-project

Part 3.b is :

- 1h of course pursuing introducing orders (partial, total), graph properties among symetry, transitivity.
- 1h of presentation and discution around **« Dynamic Carpooling Intra-Modal with Transhipment(s) » work.**

Dynamic Carpooling Intra-Modal with Transhipment(s).

- To be donwloaded separately



Dynamic Carpooling Intra-Modal with Transhipment(s).

Realized by:

- **Philippe CANALDA** philippe.canalda@femto-st.fr
- **Mohamed HASSINE** mohamedhassinef35@gmail.com
- **Idriss HASSINE** drisshassine@yahoo.fr

Subject of practical work n ° 1&2 (or homework)

- Prg1 = return the 'making change' greedy approach, taken from the course.
  The initialization of the data is carried out "hard", that is to say by using int L[7] = {5, 2, 1, 0.5, 0.2, 0.1, 0.05};
  The preferred language is JAVA. It could be C, or C ++.
  - » As support for your work, you must provide, even in a draft version: the class diagram, the factory fab (int) which updates the instantiation of the test, the compilation command, the command or the script execution (s), the results files, an analysis of these results and their reviews.
- Prg2 = exhaustive calculation of all solutions with (or without) recursivity. The calculation is long. Mostly printing to the screen or to a file. All the solutions are displayed on the screen with an overload of the toString (...) function. The result can be redirected to a solutions_n ° test_output.txt file

Subject of practical work n ° 1&2 (and homework)

- Work to be done in practical work continued: part giving change continued
  - Prg2 = exhaustive calculation by recursion (also called dynamic programming algorithm) where all the solutions are generated in an in-depth walk. The solution of the current calculation is stored in an array of the same length as L and which contains the number of units of the available values. The combinations are displayed on the screen as they are being computed (when a computed solution is valid, respectfull of all constraints). This approach makes it possible to avoid the consumption of allocated memory (also known as RAM). However, I / O are special files called buffers which consume resources (slowness).

Suite TP1 - change part continued

- Continuation Prog 2: The data displayed on the screen will be redirected to a file AllTheSolutionsTest1.txt
  When this file is validated (verified by you) you suffix it with CORRECT_
  You can restart the java execution Prg1> AllTheSolutionsTest1.txt You will compare the 2 files with the command :
  diff CORRECT_AllSolutionsTest1.txt AllSolutionsTest1.txt
- Prg2_bis: from prg 2 write Prg_2bis which reverses the order of the generation of solutions by favorizing the maximum number of units of the value being processed.
- Prg3 = You calculate all the solutions that you do not display, as you go. If you store the solutions in a 2D array of solutions, in the order in which the valid combinations are found, then a subsequent display following that order can be compared with the contents of the CORRECT_ ... file and will show no difference.

Continuation of suiteTP1

- Prg4 = modify program 2 so as to evaluate the cost of a solution according to the number of units of the values. Calculate the best solution according to this cost by displaying successive values that improve. Display the trace of this execution in a file CORRECT_SolutionsWhichImproveIncrementally.txt
  Calculate the number of solutions displayed, and the display economy, using the wc –l command. Calculate with the time, or date command, the execution time programs 2 and 4.
- Prg5 = modify program 4 or program 3 so that the best calculated solution is stored in a solution table. Evaluate the computation time saved and the display savings.
- Prg6 = from program 5, implement the cut. To do this, the solution cost value being calculated must be calculated as you go. When the “partial” cost is greater than the best_cost, you stop recursive processing. This amounts to adding an end (if then else additional to the recursive processing).

Continuation of suiteTP1

- program 6 bis: How effective is the cut, and/or the gain, when the order implemented in prg3bis is implemented in prg 6?
- Program 7 that you could do but that I don't ask: is equivalent to calculating a sorted subset of the 4 best solutions.
- Program 7' which you could do but which I don't ask: is equivalent to calculating the best solution as well as the 2 less worst solutions. The first of the less worse solutions will have a better cost but for an amount to be made positive. The second less worse will have a better cost but for an amount to be made negative. Would you be able to realize this program !!!!!

– Relationship part

- The Boolean function is Transitive (int Relation (int [] [])) which evaluates whether a relation is transitive or not. The relation passed in parameter is the most simplistic with t [i] [j] taking the values -1 if no relation, 0 on the diagonal, and a positive value of cost to characterize the trsnision of the node i towards the node j.

Continuation of MSC project

- Project activity
  - 2 functions to be developed here f1 and f2. the incomplete displacement graph can be obtained by interrogating a GIS. The matrix is somewhat sparse because the paths obtained by transitivity are not initialized => a saving of time, access to the GIS and therefore a saving of money when the number of calls is counted. An example of Graph g not complete thus :

    g = 0 1 8 -1
    -1 0 5 2
    1 -1 0 -1
    3 2 -1 0

    f1 consists in making the transitive closure. We traverse all the arcs of g: t [i] [j]. If t [i] [j]> 1 I go looking for all the t [j] [k]> 0. For such a couple t [i] [j] + t [j] [k] if it is > t [i] [k] existing, I replace this t [i] [k].

Continuation of MSC project

- f1 pursued
  - It is Dijkstra's algorithm that calculates the shortest paths from any point to any point.
    Warning ! There is a degree of difficulty between isTransitive and this Transitive closure. During a complete walk over the matrix roll, if a modification occurs, and therefore an addition of edge is issued from the transitivity computation, it is necessary to redo a complete walk, in search of an improvement of the transitive closure. We stop when there has been no improvement or modification during the complete walk over the matrix roll.

Continuation of MSC project

- f2 :
  - The f2 function uses a list of people positions. One then determines the point within the list of all the (discrete) positions of the problem by calculating the total value of all the displacements of the people to converge towards this point.

    We keep the point that minimizes the overall movements of the people of the group of students using the application.
    In order to, then, be able to generate the routes of each one, it is sufficient to modify the time matrix into a matrix of cells. If the cell is null, it matches -1. Otherwise, the cell will be made up of a cost in time, or in km, or a more complex table depending on the mode (bike, foot, car, etc.) and the time of day for taking congestion into account.

Continuation of MSC project

- f2 pursued:
  - … This cell is also made up of the route in text format which explains how to go from i to j. Going from i to k through j will correspond to the concatenation of the route t [i] [j] with that t [j] [k]. And that”s all folk!

    You have the innovative elements that many mobility solutions have not yet implemented;)

Part 3.b

- The challenge of the TD work to follow this presentation session will be :
  - for a given example with L={5, 2, 1, .5, .2, .1, .05} and m=12.35
  - greedy algorithm
    - » how much execution time ? How much print screen memory is used ?
  - exhaustive and enumerated version algorithm
    - » how much execution time ? How much print screen memory is used ? How much dynamic memory is used to keep in mind the best solution(s) ?
  - cut and price and share version
    - » how much execution time ? How much print screen memory is used ? How much dynamic memory is used to keep in mind the best solution(s) ? How much we have shared dynamic memory ? How much cuts are done ?

- What are these relational elements for ?
  (ditto for orders)

- First of all we characterize
- then, very often, it allows to reduce tests and calculations (operational)
- and then, very easily, it will be provable or certifiable (theoretical, fundamental)

- Practically, when Reflexive, symetrical and transitive we build the Hasse diagram and we do a dual treatment on a simplified object => breaks the complexity remarkably

Dual treatment

- E1xE2x…xEn ----------→ E1xE2x…xEn

  R^n et f_{R^n}

Construction of the Hasse Diagram

^ semantical

| adding

|

f_s{R^n}

E1xE2x…xEn ----------→ E1xE2x…xEn

- Element of complexity
  If the complexity of a problem is 6^N
  with N the size of a list of elements
  then give the array of values when N = 1, 2, 3, 10, 20

- Now if N can be divided by 2 (if symetrical), by 6 if (symetrical, reflexive and transitive), give the new series of valus of complexity

- Property, graph observation, and then heuristics: to find circular graphs (or subgraph) with a connected component
  - Test if irrefexive
  - Intransitive
    and asymmetric

Remark : usable to find rings in a territory

- Program
  - Present
    - The sequence of ordered structures
    - Trellises and applications
    - Boole's algebra
    - Exercises

    - Elements of complexity (algorithms and problems).
    - Exercises.
    - 2. Graphs and scheduling in project management
    - Reminders of elementary concepts of graph theory:
      - general definitions, oriented graphs, non-oriented graphs, some particular graphs, representation of graphs, browsing in a graph, exercises.
      - …

Chapter Basics : Preorder, equivalence, order

- A preorder relation is a reflexive and transitive relation
- An equivalence relation is a reflexive, symmetrical and transitive relation
- A (broad) order relationship is reflexive, antisymmetric and transitive
- Example of the criterium of champions
  - Champions are ranked according to the relationship of "ranking better or as good as":
    - 1st Camille,
    - 2nd tie Anatole and Désiré,
    - 4th Ernest
    - 5th Bernard
  - Give the representation in the form of an array with A = Anatole, B = Bernard,…, E = Ernest

Example of the criterium of champions

- Champions are ranked according to the relationship of "ranking better or as good as":
  - 1st Camille,
  - 2nd tie Anatole and Désiré,
  - 4th Ernest
  - 5th Bernard
- Give the representation in the form of an array with A = Anatole, B = Bernard,…, E = Ernest

(a,a)				
	(b,b)			
		(c,c)		
			(d,d)	
				(e,e)

reflexive

Example of the criterium of champions

- Champions are ranked according to the relationship of "ranking better or as good as":
  - 1st Camille,
  - 2nd tie Anatole and Désiré,
  - 4th Ernest
  - 5th Bernard
- Give the representation in the form of an array with A = Anatole, B = Bernard,…, E = Ernest

(a,a)	(a,b)		(a,d)	(a,e)
	(b,b)			
(c,a)	(c,b)	(c,c)	(c,d)	(c,e)
(d,a)	(d,b)		(d,d)	(d,e)
	(e,b)			(e,e)

reflexive non-symmetrical,
and transitive,
is not anti-symmetric

Some details to achieve previous exercises : Case of du calculation of intransitivity

- Algorithm for determining the transitivity of a relation
  - Input = a 2-dimensional array
  - I scan the i rows and then the j columns of each row
    and for each x, y element of the row I scan all the elements of the y row
    - All these elements of line y, I aggregate them to x, y
      and I check that for each y, something, that I therefore associate with x, y, then there is x,something

I check that all the pairs satisfy the transitivity. As soon as one does not satisfy, I stop the treatment and the answer will then be “not to be transitive”.

Some details to achieve previous exercises : Case of du calculation of intransitivity

- Same as above except that for any pair x,y and y,z,
  I check that x, z is not in the relation.

Example of an equivalence relationship

- Let us consider the relation “to be parallel to or coincident with”, for the 2 following groups of parallel lines: A || C and B || D || E.
  - Give the representation in the form of an array
- Considering the example of the criterium, determine the equivalence classes

(a,a)		(a,c)		
	(b,b)		(b,d)	(b,e)
(c,a)		(c,c)		
	(d,b)		(d,d)	(d,e)
	(e,b)		(e,d)	(e,e)

reflexive, symmetric,
and transitive
2 classes {A, C} and {B, D, E}

- Interesting exercise:
  - How will you calculate the equivalence classes of a relation?
    - Write the program.
    - What gain in complexity can we expect?
- Can you give an example of professional applications? (telephony)

Towards a strict order ...

- It is obvious that {A, D} forms an equivalence class. And that's pretty much “the same rank as relationship”.
- By making the quotient of the pre-order by this equivalence relation we obtain 4 classes which are ordered strictly 2 to 2 according to the relation "to have a better rank than":
  - The 4 classes {C}, {A, D}, {E} and {B}
  - With {C}> {A, D}> {E}> {B}
- Reorder the table according to the ascending rows C, A, D, E and B
- Give the table of the strict relation on the classes
- Qualify this relationship and notice that it is irreflexive, asymmetric and transitive

(a,a)	(a,b)		(a,d)	(a,e)
	(b,b)			
(c,a)	(c,b)	(c,c)	(c,d)	(c,e)
(d,a)	(d,b)		(d,d)	(d,e)
	(e,b)			(e,e)

An order in the broad sense (i.e. less strict)

- Reminder: It is reflexive, anti-symmetrical and transitive
- Let N * and the relation y | x (x divides exactly y, without remainder / or again y modulo x == 0).
  - Show that the relation is an order in the broad sense
  - Let X be a subset of N * such that X = {1, 2, 3, 5, 10, 20, 30}
    Calculate the table representing x | y on X

Build relation table and relationnal Graph

Exam question

- For a relation, a diagram or a graph, determining the properties of reflexivity, symmetry, transitivity, equivalence classes, this allows to reduce the starting data. It also helps to reduce the set of good solutions (targets). For each characteristic, it is necessary to be able to estimate the saving in quantity (size of the initial data, complexity saved by the algorithm, savings on the calculated solutions).
- For a given R relation, you need to formulate the new problems and evaluate the savings. We will have to deduct from these savings, possibly the additional calculation of the elements of an equivalence class, the calculation of the solutions resulting from a symmetry that we have taken into account to reduce the diagram of the starting data, etc.

- This relation is upper triangular (TU) => we can do without studying the lower part (TL).

- Likewise, the route TL is avoided when the relation is symmetrical. We calculate TL and the solution list. Processing the TL part amounts to duplicating ls2TU and applying symmetry to obtain ls2TL. The gain is known: if n is the size of ls2TU, and N the size of R, the gain is (NxN) / 2 - n

Partial and total orders

- Reminder: It is reflexive, anti-symmetrical and transitive
- Let N * and the relation y | x (x divides exactly y, without remainder / or even x modulo y == 0).
  - Show that the relation is an order in the wide sense
  - Let X be a subset of N * such that X = {1, 2, 3, 5, 10, 20, 30}

    Calculate the table representing x | y on X

(1,1)	(1,2)	(1,3)	(1,5)	(1,10)	(1,20)	(1,30)
	(2,2)			(2,10)	(2,20)	(2,30)
		(3,3)				(3,30
			(5,5)	(5,10)	(5,20)	(5,30)
				(10,10)	(10,20)	(10,30)
					(20,20)	
						(30,30)

The reflexivity is visible,
the anti-symmetry too,
the transitivity a little less

all the elements are not comparable
below the diagonal

Partial and total orders

- The order relationship is partial when all the elements are not comparable 2 to 2
- Otherwise it is total

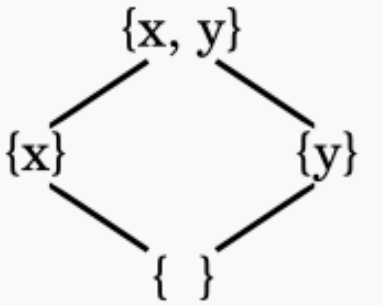
Sagittal representation and Hasse diagram

- In a sagittal representation
  - 2 elements a and b are each represented by 1 bullet,
  - they are joined by an oriented edge a - b if a R b
    - Example if a | b
      there are as many edges as there are pairs in the array representation of the same relation
- In the Hasse diagram, the edges that result from reflexivity and transitivity are removed

  - Give these 2 representations of the relation "|" on X
  - List 3 representations: array, sagittal diagram, Hasse diagram
  - Give the algorithms that produce the last 2 representations from the 1st representation (the 2-dimensional array)

Hasse Diagram

In order theory, a **Hasse diagram** (/ˈhæsə/; German: [ˈhasə]) is a type of mathematical diagram used to represent a finite partially ordered set, in the form of a drawing of its transitive reduction. Concretely, for a partially ordered set $(S, \leq)$ one represents each element of S as a vertex in the plane and draws a line segment or curve that goes *upward* from one vertex x to another vertex y whenever y covers x (that is, whenever $x \neq y$, $x \leq y$ and there is no z distinct from x and y with $x \leq z \leq y$). These curves may cross each other but must not touch any vertices other than their endpoints. Such a diagram, with labeled vertices, uniquely determines its partial order.



The power set of a 2-element set ordered by inclusion

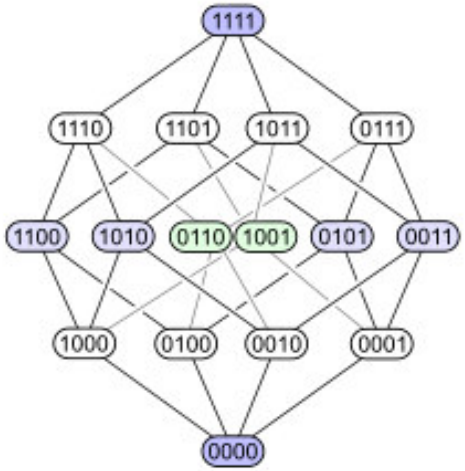
Hasse diagrams are named after Helmut Hasse (1898–1979); according to Garrett Birkhoff, they are so called because of the effective use Hasse made of them.[1] However, Hasse was not the first to use these diagrams. One example that predates Hasse can be found in an 1895 work by Henri Gustave Vogt.[2][3] Although Hasse diagrams were originally devised as a technique for making drawings of partially ordered sets by hand, they have more recently been created automatically using graph drawing techniques.[4]

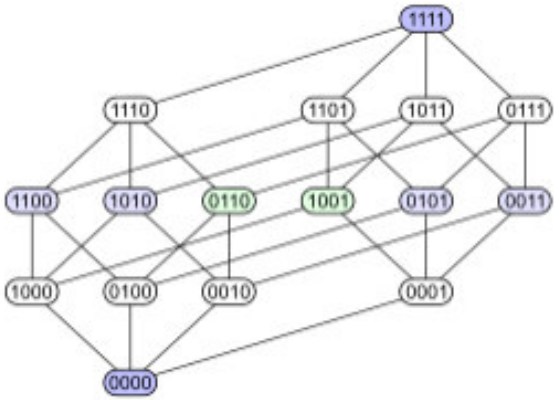
In some sources, the phrase "Hasse diagram" has a different meaning: the directed acyclic graph obtained from the covering relation of a partially ordered set, independently of any drawing of that graph.[5]

Design Hasse Diagram

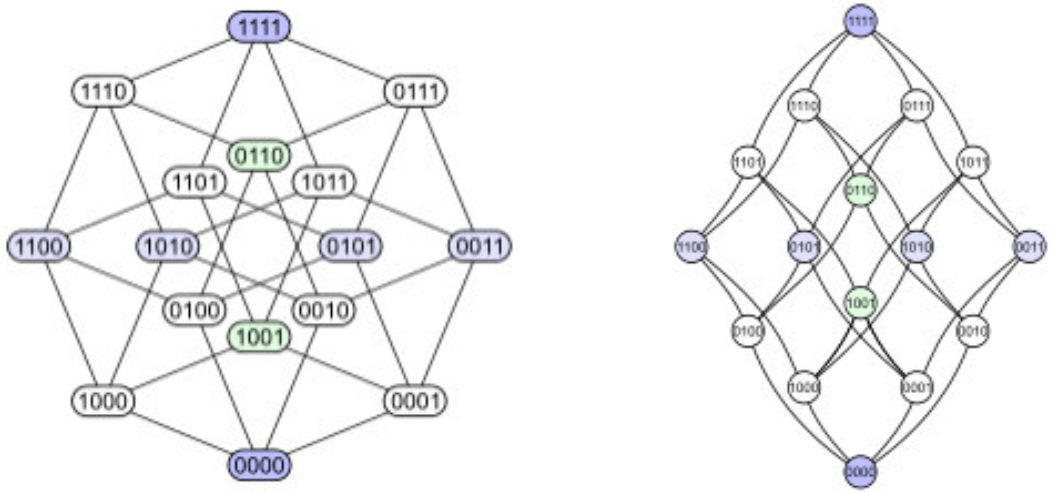
Although Hasse diagrams are simple, as well as intuitive, tools for dealing with finite posets, it turns out to be rather difficult to draw "good" diagrams. The reason is that, in general, there are many different possible ways to draw a Hasse diagram for a given poset. The simple technique of just starting with the minimal elements of an order and then drawing greater elements incrementally often produces quite poor results: symmetries and internal structure of the order are easily lost.

The following example demonstrates the issue. Consider the power set of a 4-element set ordered by inclusion $\subseteq$. Below are four different Hasse diagrams for this partial order. Each subset has a node labelled with a binary encoding that shows whether a certain element is in the subset (1) or not (0):





Design Hasse Diagram



The first diagram makes clear that the power set is a graded poset. The second diagram has the same graded structure, but by making some edges longer than others, it emphasizes that the 4-dimensional cube is a combinatorial union of two 3-dimensional cubes, and that a tetrahedron (abstract 3-polytope) likewise merges two triangles (abstract 2-polytopes). The third diagram shows some of the internal symmetry of the structure. In the fourth diagram the vertices are arranged in a 4×4 grid.

- The exercise of transforming the sagittal matrix into a Hasse diagram is VERY interesting.
- It exploits the ordering of the elements of N * manipulated.

- Program
  - Present
    - The particular elements of an ordered set
    - Trellises and applications
    - Boole's algebra
    - Exercises

    - Elements of complexity (algorithms and problems). Exercises. 2.
    - Graphs and scheduling in project management
    - Reminders of elementary concepts of graph theory:
      - general definitions, oriented graphs, non-oriented graphs, some particular graphs, representation of graphs, browsing in a graph, exercises.
      - ...

Some properties ...

- To obtain a representation of a strict order from a wide order (reflexive, antisymmetric and transitive) it is necessary ...
- … you have to remove the diagonal.

Rmk :

1 a reflexive, asymmetric and transitive relation defines a strict order.

2 it can induce a partial order or a total order

- Example:
  - represent the strict relation corresponding to '||' which means divides (in the Euclidean way) and is not equal. Is the order obtained strict or partial? Justify.

Some properties ...

- To any relation <= (or <) corresponds a converse relation> = (>, respectively)

  Question: How would you get this converse relation?

- Symmetry from the diagonal

The particular elements of an ordered set (1/3)

- **Definition:** a set on which an order relation (partial or total) is defined is said to be (partially or totally) ordered.
- X a set partially ordered by the relation <=, and a part P of X
  - A lower bound m of P is an element m of X which is <= any element x of P
    - Similarly, an upper bound M of P is an element M of X which is> = any element x of P
  - A maximal element of P is an element M of P for which there is no element of P> to M
    - Similarly, a minimal element of P is an element M of P for which there is no element from P <to M

The particular elements of an ordered set (2/3)

- The largest element E (maximum) of P is such that, E belongs to P and for all x of P we have E>= x
  - Similarly, the smallest element e (minimum) of P is such that, e belongs to P and for all x of P we have e <= x
- The upper bound (u.b.) B (supremum) of P, also noted supP is the smallest element of the set of upper bound of P.
  - Similarly the lower bound (l.b.) b (infimum) of P, also noted infP is the largest element of the set of lower bound of P.
- The universal element of X is the largest element of X.
- The null element of X is the smallest element of X.

- Rmk : all these elements do not necessarily exist, due in particular to the fact that sometimes the elements are not comparable.

The particular elements of an ordered set Example (3/3)

- Let X = {1, 2, 3, 5, 10, 20, 30}, be a set partially ordered by the relation x | y (x divides y).
  For the following two parts,
  - P_1 = {2,3,5,10} and P_2 = {2,5,10}
    - qualify them
      - the upper bound of P,
      - the lower bound of P,
      - the maximum elements
      - and the minimum elements,
      - the largest element of P,
      - the smallest element of P,
      - the u.b., the l.b.,
      - the universal element
      - and the null element.

- 1/ TD solve exercise slide 43,
- 2/ the complexity evaluation of making change
- 3/ adapt the transitive closure, or is-transitive or the best Mo-d which minimizes the path between 2 points of a graph,
  to
  remove the transitive edges of a relation that means that, for any t[x][z], having t[x][y] and t[y][z], remove t[x][z] from the relation

Open problem:

- If I associate a position in 2D space (resp. 3D) with each node (1, 2, 3, etc.) and each relation results in a directed segment connecting 2 nodes,
  i) does there exist the possibility of arranging nodes and edges without any edge cutting?
  ii) can you calculate the best arrangement of the nodes of the relation so that the number of intersecting edges is minimized?

The particular elements of an ordered set (synthesis)

- **Definition:** a set on which an order relation (partial or total) is defined is said to be (partially or totally) ordered.
- X a set partially ordered by the relation <=, and a part P of X
  - A lower bound m of P is an element m of X which is <= any element x of P
    - Similarly, an upper bound M of P is an element M of X which is> = any element x of P
  - A maximal element of P is an element M of P for which there is no element of P> to M
    - Similarly, a minimal element of P is an element M of P for which there is no element from P <to M
  - The largest element E (maximum) of P is such that, E belongs to P and for all x of P we have E> = x
    - Similarly, the smallest element e (minimum) of P is such that, e belongs to P and for all x of P we have e <= x
  - The upper bound (b.s.) B (supremum) of P, also noted supP is the smallest element of the set of upper bound of P.
    - Similarly the lower bound (l.b.) b (infimum) of P, also noted infP is the largest element of the set of lower bound of P.
  - The universal element of X is the largest element of X.
  - The null element of X is the smallest element of X.

The particular elements of an ordered set Example (3/3) first result

- Let X = {1, 2, 3, 5, 10, 20, 30}, be a set partially ordered by the relation x | y (x divides y).
  For the following two parts,
  - P_1 = {2,3,5,10}
    - qualify them
      - the upper bound of P, {30}
      - the lower bound of P, {1}
      - the maximal elements, {3, 10}
      - and the minimal elements, {2, 3, 5}
      - the largest element of P, inexistant
      - the smallest element of P, inexistant
      - the supremum, {30}
        the infinimum, {1}
      - the universal element
      - and the null element, {1}.

The particular elements of an ordered set Example (3/3) second result

- Let X = {1, 2, 3, 5, 10, 20, 30}, be a set partially ordered by the relation x | y (x divides y).
  For the following two parts,
  - P_2 = {2, 5,10}
    - qualify them
      - the upper bound of P, {10, 20, 30}
      - the lower bound of P, {1}
      - the maximal elements, {10}
      - and the minimal elements, {2, 5}
      - the largest element of P, {10}
      - the smallest element of P, inexistant
      - the supremum, {10}
      - the infinimum, {1}
      - the universal element, inexistant
      - and the null element, {1}.